Sovereign Compute Network: The Time Machine Model

TL;DR

Application state is not stored—it's computed.

Every event is persisted with a cryptographic hash. The entire app memory/state is **derived on-demand** by replaying events through a pure function. You can instantly query any point in history.

```
Events (indexed, hashed, persisted) → Pure deriveState() → App state
                    ↓
            (No separate DB!)
                    ↓
        Everything derived on-the-fly
```

The Model

Traditional Architecture

```
User Action → API → Database (writes state) → Database (reads state) → UI
                          ↑
                  "Source of truth"
             (separate from audit log)
```

Problems:

- State and audit trail can diverge
- Rebuilding history requires replaying logs separately
- Recovery means restoring from backups
- No easy way to query "what was state at time T?"

Sovereign Architecture

```
User Action → emit(event) → IndexedDB (stores event) → deriveState(events) → UI
                                                ↑
```

```
                    Source of truth
                (no separate DB needed)
```

Advantages:

- State is **always correct** (derived from immutable log)
- **Perfect audit trail** (log = whole history)
- **Instant time travel** (query any historical state)
- **Tamper detection** (hash chain proves integrity)
- **Offline-first** (derive state locally, sync async)
- **Zero data loss** (log is persisted to IndexedDB)

How It Works

Step 1: Event Emitted

```javascript
sovereignLog.emit({
  type: 'MODEL_SELECTED',
  model: 'llama3'
});
```

Step 2: Event Persisted

```javascript
{
  type: 'MODEL_SELECTED',
  model: 'llama3',
  seq: 42,
  ts: 1709251200000,
  hash: 'a1b2c3d4...',  // FNV32(payload | prevHash)
  prevHash: 'z9y8x7w6...'
}
// ↓ Frozen and saved to IndexedDB
```

Step 3: State Derived

```javascript
function deriveState(eventLog) {
  let state = {
    model: null,
    messages: [],
    ...
  };

  for (const event of eventLog) {
    // Apply event to state (pure, no side effects)
    if (event.type === 'MODEL_SELECTED') {
      state.model = event.model;
    }
    if (event.type === 'MESSAGE_ADDED') {
      state.messages.push(event.message);
    }
  }

  return state;
}

// Current state
const currentState = deriveState(eventLog);

// Historical state (50 events ago)
const oldState = deriveState(eventLog.slice(0, eventLog.length - 50));

// Any point in time
const stateAtSeq100 = deriveState(eventLog.slice(0, 100));
```

Step 4: On Page Load

```javascript
```

```
// Page reload:
1. Load eventLog from IndexedDB
   (IndexedDB.load('sovereign-log'))

2. Verify hash chain (tamper detection)
   for each event:
     if hash(event.payload | prevHash) !== event.hash:
       throw Error('Log corrupted')
     prevHash = event.hash

3. Compute state from verified log
   state = deriveState(verifiedLog)
   // Full app reconstructed in <100ms
   // Zero data loss
   // Cryptographically proven integrity

4. Resume UI
   subscribers notified of state
   (same state as before reload)
```

Use Cases

1. Time-Travel Debugging

```javascript
// User: "What was happening at event 847?"
debugState = deriveState(eventLog.slice(0, 847));
console.log(debugState);  // Historical state, verified

// User: "Take me back to 30 seconds ago"
const target = eventLog.find(e => e.ts > Date.now() - 30000);
undoState = deriveState(eventLog.slice(0, target.seq));
```

2. Offline-First Applications

```
Online: Persist events → gossip to peers → finality
Offline: emit(event) → persist to IndexedDB
```

```
On reconnect: gossip log → peers catch up → sync finalized

Throughout: deriveState(localLog) keeps app working
```

3. Collaborative Tools

```
User A: edits doc → emit(EDITED) → log + gossip
User B: (offline) edits same doc → emit(EDITED) → local log

On sync:
  merge logs deterministically (sort by hash)
  both derive from merged log
  → same final state (conflict-free!)
```

4. Audit & Compliance

```javascript
// Full audit trail with cryptographic proof
const auditLog = eventLog.map(e => ({
  seq: e.seq,
  type: e.type,
  hash: e.hash,
  prevHash: e.prevHash,
  timestamp: e.ts
}));

// Prove integrity
verifyChain(auditLog);  // Throws if tampered
// → Suitable for regulatory compliance (financial, healthcare)
```

5. Undo/Redo Stack

```javascript
```

```
// Undo: derive from eventLog without last N events
undoState = deriveState(eventLog.slice(0, -1));

// Redo: restore full log
redoState = deriveState(eventLog);

// Multiple undo/redo levels are free (pure function)
```

Performance Characteristics

Derivation Cost

- **First load:** O(n) where n = event count
  - 1000 events: ~50ms
  - 10,000 events: ~500ms
  - 100,000 events: ~5s

  (Linear because deriveState must visit every event)

Optimization: Snapshots

```javascript
```

```
// Every 1000 events, save checkpoint
if (eventLog.length % 1000 === 0) {
  snapshot = {
    height: 1000,
    state: deriveState(eventLog),
    stateHash: hash(state)
  };
  IndexedDB.save('snapshot-1000', snapshot);
}

// Recovery: load snapshot + replay suffix
snapshot = IndexedDB.load('snapshot-1000');
suffix = eventLog.slice(1000);
state = deriveState([...snapshot.log, ...suffix]);
// Much faster for large logs
```

Memory Usage

```
Traditional: DB (full state) + index + cache + log
Sovereign:  Log only
            (derive state on-demand)
            (memory freed after use)
```

Comparison with Industry

Redux (Meta)

- Actions logged, but only for devtools
- State stored separately in JS memory
- Time travel requires manual implementation
- No built-in persistence

Event Sourcing (DDD)

- Similar idea (log = truth)
- Typically server-side (DB-backed)

- Recovery = replay from DB
- Sovereign: Client-side, IndexedDB, instant

Blockchain

- Similar persistence model (chain = truth)
- Consensus overhead (10 seconds finality)
- Sovereign: Deterministic ordering (instant), quorum finality (milliseconds)

Why Sovereign's Approach Wins

Aspect	Traditional DB	Event Sourcing (server)	Blockchain	Sovereign
Source of truth	Mutable state	Immutable log	Immutable log	Immutable log
Audit trail	Separate (lossy)	Built-in	Built-in	Built-in
Time travel	Snapshots only	Replay whole log	Replay chain	Instant (any point)
Tamper proof	No	No	Yes (crypto)	Yes (hash chain)
Environment	Server	Server	Network	Browser
Offline	No	No	No	Yes
Latency	Network-bound	Network-bound	Network-bound	<100ms
Determinism	Non-deterministic	Non-deterministic	Consensus-driven	100% deterministic

---

The Magic

The key insight: **If your `deriveState()` function is pure (no side effects, no timestamps, deterministic), then:**

1. All nodes with the same event log derive **identical state** (no divergence)
2. Replaying from any point gives **correct historical state** (time machine)
3. Hash chain proves **nothing was tampered** (audit-proof)
4. Works **offline** and **online** (same logic)
5. No server needed to run the application

This is what makes Sovereign different.